Java Collections Guide

1. HashMap

HashMap stores key-value pairs in a hash table. It uses the key hash to choose a bucket, then compares keys using equals when resolving collisions.

In Java 8 and later, heavily populated buckets can be transformed into balanced trees under implementation-specific conditions. HashMap allows one null key and multiple null values. It is not thread-safe.

The default load factor is 0.75. Resizing increases table capacity when the number of entries exceeds the threshold. A good hash distribution helps lookup, insertion and removal approach constant average time.

Java Collections Guide

2. ConcurrentHashMap

ConcurrentHashMap supports concurrent access by multiple threads. In modern Java implementations, updates use compare-and-set operations and synchronization of individual bins when needed.

ConcurrentHashMap does not permit null keys or null values. It offers atomic operations such as putIfAbsent, compute and merge. These operations avoid a separate check-then-act race.

Iterators are weakly consistent. They do not throw ConcurrentModificationException solely because another thread updates the map. A synchronized HashMap requires external locking for compound operations.

Java Collections Guide

3. ArrayList and LinkedList

ArrayList stores elements in a resizable array. Access by index is constant time. Appending is amortized constant time, while insertion in the middle usually shifts elements.

LinkedList stores elements in linked nodes. Finding an element by index requires traversal. Adding or removing through a positioned iterator changes nearby links, but reaching that position still has a cost.

ArrayList is often preferred for iteration and random access. Choose a data structure from measured access patterns rather than assuming linked nodes always make insertion faster.